25.1 Hash Table Recap, Default Hash Function

Let's continue understanding Hashing. We've now seen implementations for sets and maps.

1. Red-Black Based Tree Approach: TreeSet/TreeMap
   - requires the items to be comparable (the notion of less or greater than)
   - logarithmic time complexity
2. HashTable based approach: HashSet/HashMap
   - constant time operations if the hashCode spreads the item nicely (few collisions)

Recall that with a hash table, the idea is that for any piece of data, like a String (or any other type of object) we want to store in our hash table, we need to turn this into a number called a hash code. So a string like "Mihir" can be converted to -2101281024.

{% hint style="info" %} Fun Fact: And that integer is anything between about negative two billion and about two billion, the space of all Java Integers. This range yields about 4 billion integers or 2^32 integers. If you are interested in why this is the case, please take CS 61C! {% endhint %}

Once we have our number, we want to convert this number to our bucket number (i.e. which of the many linked lists I want to add this new entry to). In the first example, we will use the`Math.floorMod(x, 4)`, since the length of the underlying buckets array has length 4.

If we take the converted hash code for "Mihir", -2101281024, and then mod this by 4, we get 0. This reduces our hash code, -2101281024 to a valid index, 0. This means that we would use the 0th bucket to place our data, "Mihir", in our LinkedList at the 0th bucket.

> You can essentially think of Java HashTables as just an array of LinkedLists categorized under bucket labels and hopefully have better performance by utilizing these LinkedLists.

But how many LinkedLists/Buckets should we use? This is an important question, because as we insert more and more items into our buckets, the length of the LinkedLists will inevitably grow, which in turn compromises the runtime for the hash table's operations.

For example, let's say we inserted strings like "Mihir", "Mirchandani", "loves", and "61B", all of which yielded hash codes that were divisible by 4. In this case, all strings would be put into the 0th index bucket and all strings would be inserted into the same LinkedList. What happens to the runtime for a search operation? Well, we would have to search through an entire LinkedList to look up our data! This runs in linear time and is too slow for HashMap's famous title of holding fast lookup times.

Last time, we saw that we can have a variable number of LinkedLists. The idea is that we resize that array of linked lists whenever the load factor (N/M, where N is the number of elements in our table and M is the number of buckets) exceeds some constant. Java picks 0.75 as we'll see later. This prevents collisions from happening too frequently. So long as our items are spread out nicely, between the buckets, the LinkedLists at each bucket for the most part have a very small size, which means we'll on average get constant run time!

Example: If the HashTable has load factor 3, and our `hashCode()` function spreads out the entries evenly, we are going to end up with just 3 items per bucket, and our search operation takes $\Theta(1)$time.

Comparing Data Structure Run Times!

	contains(x)	**add(x)**
Bushy BSTs	Θ(log N)	Θ(log N)
Separate Chaining Hash Table with NO resizing	Θ(N)	Θ(N)
Separate Chaining Hash Table with resizing	Θ(1)	Θ(1)

25.2 Distribution By Other Hash Functions

Suppose our `hashCode()` implementation simply returns 0.

```
@Override
public int hashCode() {
    return 0;
}
```

▶ What distribution do we expect?

In order to get a more even distribution, what we can do is something to what we tried in the previous section where we utilize modulo. Let's say that we define the size of our Hash Table to have 4 buckets. This means it has 4 corresponding LinkedLists and 4 bucket indices labeled {0, 1, 2, 3}. The modding is not required in our `hashCode()` function as it is being done for us in the hash table to guarantee we can add to that bucket. As we said the hash code could really be any integer in the range of 4 billion unique values!

By using modulo, we ensure that our hashcode yields a number that can be represented as an index and clearly identifies which LinkedList to add to. Additionally, when adding a series of numbers at once, we see that we get an even distribution of numbers in our LinkedList yielding a constant lookup time.

```
@Override
public int hashCode() {
    return num;
}
```

This hash function should yield a much more even distribution! Objects with different `num` will now be more spread out across the buckets instead of all living in the 0th bucket. If our class does not explicitly override the `hashCode()` function, Java will use the default implementation, which returns the object's address in memory as its hash code!

Why Bother With Custom Hash Functions?

{% embed url="https://youtu.be/4_gRlDS9AMQ" %}

Let's discuss if the default hashCode function is a good hashCode function! It actually is a good spread as it relies on the fact that different objects will live in different places in the memory, and the memory address is effectively random. We will get a good distribution, since objects are basically assigned random indices to insert into the hash table.

This really raises an interesting question: why do we care about other custom hash functions when the default hashcode gets good spread? We'll read about this in the next section.

25.3 Contains & Duplicate Items

Contains

The `equals()` Method for a ColoredNumber Object

Suppose the `equals()` method for ColoredNumber is as below, i.e. two ColoredNumbers are equal if they have the same num.

```
@Override
public boolean equals(Object o) {
   if (o instanceof ColoredNumber otherCn) {
       return this.num == otherCn.num;
   }
   return false;
}
```

HashSet Behavior for Checking `contains()`

Suppose the `equals()` method for ColoredNumber is on the previous slide, i.e. two ColoredNumbers are equal if they have the same num.

```
int N = 20;
HashSet<ColoredNumber> hs = new HashSet<>();
for (int i = 0; i < N; i += 1) {
    hs.add(new ColoredNumber(i));
}
```

Suppose we now check whether 12 is in the hash table.

```
ColoredNumber twelve = new ColoredNumber(12);
hs.contains(twelve); //returns true
```

{% hint style="info" %} What do we **expect** to be returned by `contains`? {% endhint %}

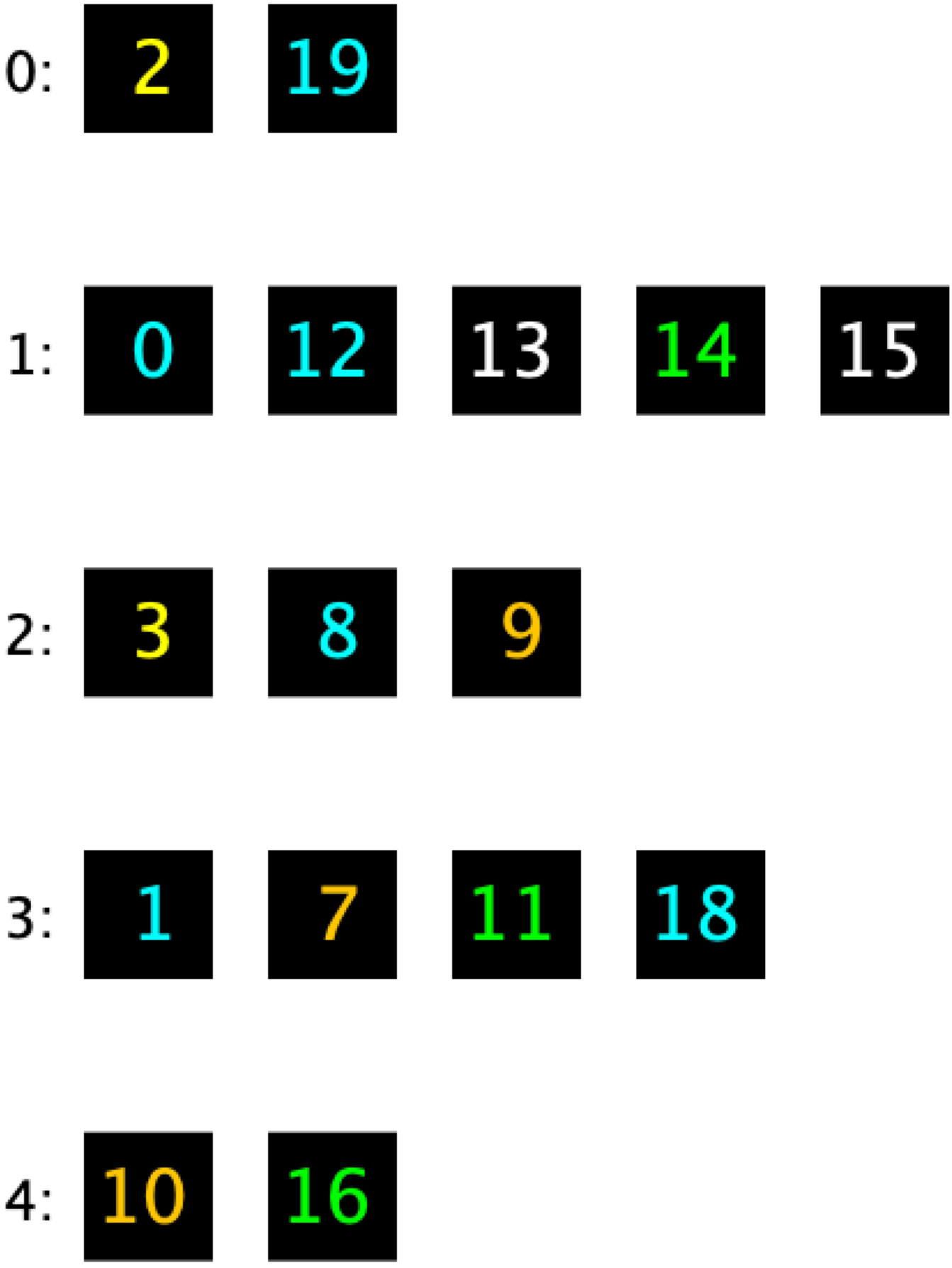
▶ Answer

Finding an Item Using the Default Hashcode

Suppose we are using the default hash function (uses memory address):

```
int N = 20;
HashSet<ColoredNumber> hs = new HashSet<>();
for (int i = 0; i < N; i += 1) {
   hs.add(new ColoredNumber(i));
}
ColoredNumber twelve = new ColoredNumber(12);
hs.contains(twelve); // returns ??
```

which yields the table below:


0:
2
19
1:
0
12
13
14
15
2:
3
8
9
3:
1
7
11
18
4:
10
16

Suppose equals returns true if two ColoredNumbers have the same num (as we've defined previously).

{% hint style="info" %} What is actually returned by `contains`? {% endhint %}

▶ Answer

{% hint style="info" %} Hard Question: If the default hash code achieves a good spread, why do we even bother to create custom hash functions? {% endhint %}

▶ Answer

Basic rule (also definition of deterministic property of a valid hashcode): If two objects are equal, they **must** have the same hash code so the hash table can find it.

Duplicate Values

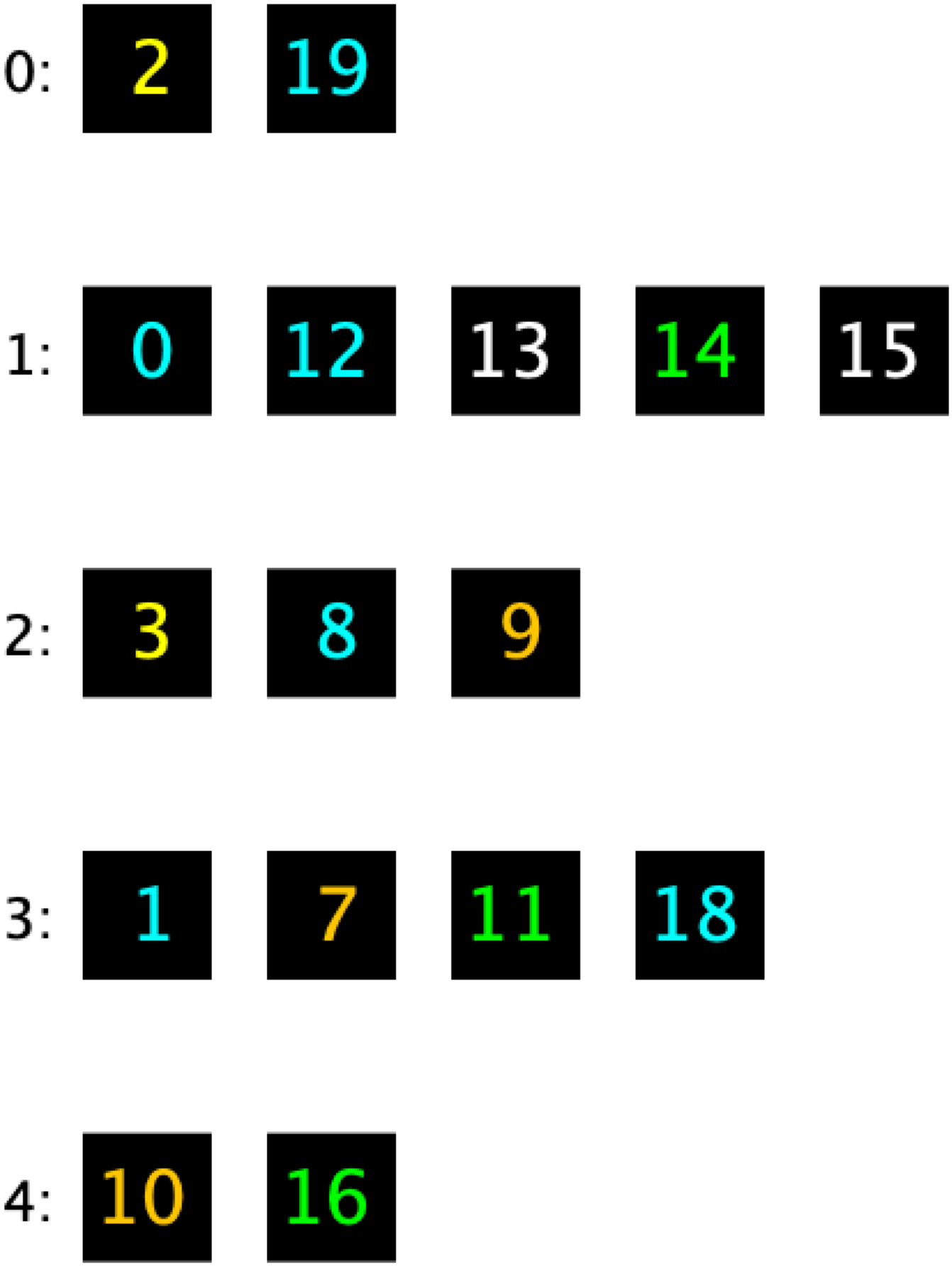
{% embed url="https://www.youtube.com/watch?index=5&list=PL8FaHk7qbOD637Q-6p7nn5dKz1tK6WAjJ&v=3HmCkRYsAGg" %}

Overriding `equals()` but Not `hashCode()`

Suppose we have the same `equals()` method (comparing `num`), but we do not override `hashCode()`.

```
public boolean equals(Object o) {
   ...  return this.num == otherCn.num; ...
}
```

The result of adding 0 through 19 is shown below:


0:
2
19
1:
0
12
13
14
15
2:
3
8
9
3:
1
7
11
18
4:
10
16


```
ColoredNumber zero = new ColoredNumber(0);
hs.add(zero); // does another zero appear?
```

{% hint style="info" %} Which can happen when we call add(zero)?

Answer Choices:

1. We get a 0 to bin zero.
2. We add another 0 to bin one.
3. We add a 0 to some other bin.
4. We do not get a duplicate zero {% endhint %}

▶ Answer

Key Takeaway: `equals()` and `hashCode()`

Bottom line: If your class override equals, you should also override hashCode in a consistent manner.

- If two objects are equal, they must always have the same hash code.

If you don't, everything breaks:

- `Contains` can't find objects (unless it gets lucky).
- `Add` results in duplicates.

25. Hashing II